Kasra:

1. WaterSplatting: System Implementation and Experimental Analysis

A Study of Dataset Construction, Training Behavior, and System Constraints

2. Introduction

WaterSplatting provides a framework for reconstructing 3D underwater scenes from multi-view imagery using a hybrid representation that combines explicit 3D Gaussian geometry with a separate volumetric water medium. Unlike standard 3D Gaussian Splatting, which renders scene geometry alone, WaterSplatting models underwater image formation by incorporating attenuation and scattering effects in the medium. Because the water is represented as a distinct field rather than integrated into the scene geometry, the method can recover both the structure of the scene and the optical properties of the water independently, making it possible to render the scene as it would appear without the water medium.

This document presents a structured analysis of the implementation and use of WaterSplatting, focusing on the reasoning behind design choices, observed system behavior, and challenges encountered during development. The goal of this work was to understand how the WaterSplatting model, introduced in *WaterSplatting: Fast Underwater 3D Scene Reconstruction Using Gaussian Splatting*, operates in practice, including the reconstruction pipeline, dataset requirements, and system behavior under different conditions. In addition, this work examines how the method generalizes to new datasets, what types of data it performs best on, and what potential improvements can be made. A broader motivation of this work was to understand the model well enough to support future integration with sonar-based splatting approaches using multimodal datasets. That integration is developed in the SonarSplat formulation presented elsewhere in this paper, which builds directly on the optical Gaussian representation studied here. (REFERENCE TO PRIYANSHU'S SECTION)

3. Problem Context

Applying WaterSplatting in practice introduces several key considerations:

- dataset image quality
- appropriate scene camera view density
- training dependence on accurate 3D point cloud reconstruction
- computational constraints during training
- variability in output quality

Most importantly, dataset preparation requires careful balancing of image count. Too many images can substantially increase reconstruction time and matching cost, while too few reduce overlap and can prevent the reconstruction from connecting different parts of a scene into a single coherent sparse model. The design choices made to address this are discussed in Section 6.

4. System Overview

The system can be understood as a sequence of transformations:

$$\text{raw dataset} \rightarrow \text{COLMAP sparse reconstruction} \rightarrow \text{Gaussian representation} \rightarrow \text{rendered output} \tag{1}$$

Each stage introduces constraints that influence further stages. COLMAP reconstructs scenes by matching image features, estimating camera poses and intrinsics, and triangulating shared 3D points into a sparse point cloud (described further in Section 6). WaterSplatting then initializes Gaussians at the reconstructed sparse geometry and optimizes Gaussian parameters jointly with a volumetric water medium (described further in Section 7). At render time, the Gaussian representation captures scene structure while the water field accounts for attenuation and scattering, enabling both underwater rendering and restored scene views.

5. Dataset Characteristics and Observations

Dataset properties have a direct impact on reconstruction and training behavior. The most important structural properties are image resolution, dataset size, consistency across frames, and the presence of artifacts or visual noise. The evaluated datasets exhibited substantial variation across these dimensions, which directly influenced sparse reconstruction quality, pose stability, training speed, and final rendering quality.

5.1. Structural Properties

Three optical datasets were especially important in evaluating WaterSplatting.

SeaThru-NeRF Dataset. The SeaThru-NeRF dataset used in this work contains 88 images across 4 scenes, occupies approximately 610 MB, is 100% PNG, and includes resolutions of 1778×1182 and 1370×913. It served as the baseline benchmark because it is the underwater benchmark used by the original WaterSplatting study and by related underwater neural rendering work.

FLSea Dataset. The FLSea dataset used in this work contains 22,877 images, spans 12 sets in two locations, occupies approximately 90 GB, and is 100% TIFF. Publicly, FLSea is described as an underwater forward-looking visual-inertial and stereo-vision dataset collected in the Mediterranean and Red Sea, with multiple stereo and visual-inertial sequences acquired under varying visibility, lighting, and structural conditions. In practice, this dataset was highly redundant across consecutive frames, which made downsampling necessary before reconstruction and training (subsampling strategy described in Section 6.2).

Sunboat Dataset. The Sunboat dataset was treated as a more difficult generalization target beyond the SeaThru benchmark. It comes from an AUV-based multi-sensor dataset and contains approximately 8,000 forward-looking camera and sonar sample pairs in its first session, recorded at depths of 10–15 m. The released optical images are downscaled to 1692×1355. In this work, the optical subset proved difficult for COLMAP due to persistent visual artifacts and inconsistent frame spacing (reconstruction adaptations in Section 6.1).

5.2. Observed Behavior

SeaThru-NeRF Dataset. SeaThru-NeRF served as the baseline benchmark because it is the dataset family used by the original WaterSplatting paper. It contained an appropriate number of images for stable reconstruction and, in practice, yielded coherent single-model reconstructions and reliable training behavior.

FLSea Dataset. FLSea produced strong results, but its scenes contained very high frame redundancy. Consecutive frames were often too similar, making full-density reconstruction unnecessarily expensive while contributing little additional geometric information. In practice, downsampling was required to make training computationally feasible without sacrificing reconstruction quality.

Sunboat Dataset. Sunboat produced weaker results and highlighted the sensitivity of the pipeline to scene artifacts and irregular frame spacing. A persistent border artifact appeared across the bottom of the images, causing sparse reconstruction to repeatedly fragment into multiple disconnected models. Because each training view included the artifact, the model treated it as scene geometry, causing it to account for the border as a persistent structure present across evaluation poses, interfering with the model's ability to connect disparate parts of the scene coherently. This failure mode motivated the development of a dedicated multi-strategy preprocessing pipeline, described in

the preprocessing section of this document, designed to detect and remove persistent border artifacts prior to reconstruction. (REFERENCING LEO'S PART)

6. Scene Representation

Not every dataset provides a pre-existing COLMAP sparse reconstruction, so a script was implemented to convert raw flat image folders into the geometric representation compatible with WaterSplatting. The main goal of this stage is to create a COLMAP sparse reconstruction, whose quality directly determines the initial Gaussian placement. The dataset observations in Section 5 motivate the specific choices described here.

6.1. Reconstruction Strategy

When a COLMAP reconstruction fails to form a single connected model, the scene fragments into multiple disconnected sparse models. To mitigate this, the matching strategy was changed from the sequential matching used as COLMAP's default to exhaustive matching. Sequential matching relies on local ordering and is efficient when neighboring images are known to overlap; exhaustive matching considers all candidate image pairs, increasing the chance of discovering connecting points across far-apart views. The tradeoff is that time cost grows quadratically with image count. This tradeoff was accepted because maximizing the probability of obtaining a single connected reconstruction was the primary objective. On the available hardware, a 600-image exhaustive reconstruction took approximately 30 minutes.

A second adaptation was to relax COLMAP's incremental mapper thresholds below their defaults. Underwater imagery presents weaker feature correspondences than typical terrestrial scenes due to textureless surfaces, backscatter, and optical artifacts, and the default thresholds are calibrated for the stronger correspondences of clear-water or above-ground data. The modifications are summarized in Table 1.

Parameter	COLMAP Default	This Implementation
`init_min_num_inliers`	100	50
`init_min_tri_angle`	4.0°	2.0°
`abs_pose_min_num_inliers`	12	10
`max_reg_trials`	3	5

Table 1: Relaxed COLMAP incremental mapper thresholds used in this implementation to accommodate weaker feature correspondences in underwater imagery.

Lowering these thresholds allows COLMAP to initialize and register cameras in scenes where textureless surfaces, backscatter, and artifacts reduce the number of reliable feature matches below what would be acceptable in clear-water or terrestrial environments. The increased `max_reg_trials` gives the mapper more attempts to register each image before discarding it, further reducing fragmentation risk.

When multiple disconnected models were still produced, `colmap model_merger` was considered as a postprocessing step. Model merging is only appropriate when disconnected reconstructions share common registered images, and any merged model should be refined afterward with bundle adjustment. Because disconnected submodels arise precisely when feature overlap is weak, merging has limited applicability in the cases where it is most needed (limitations discussed further in Section 10).

6.2. Design Considerations

Several practical design considerations follow from this behavior. First, image overlap must be sufficient to ensure a single connected reconstruction. Second, dataset cleanliness significantly impacts reconstruction quality, because repeated artifacts can create misleading matches that do not support valid scene geometry (as observed with the Sunboat dataset in Section 5). Third, subsampling strategies are required to balance reconstruction feasibility, scene coverage, and computational cost. The scene-construction script was designed around these tradeoffs, allowing

conversion from a flat folder of images into a WaterSplatting-ready structure with generated `poses_bounds.npy` and a COLMAP sparse reconstruction.

Reproducibility note: Different combinations of subsampling and evaluation rates work best per scene, so the scene-construction script exposes two parameters that must be tuned per dataset: the frame subsampling rate (which controls how many frames are used from the source sequence) and the evaluation split interval (which controls what fraction of frames are held out for validation). Neither has a meaningful universal default — appropriate values depend on the density and redundancy of each individual scene.

7. Training Dynamics

7.1. Initialization and Training Loop

WaterSplatting initializes Gaussians from the COLMAP point cloud (Section 6) by placing blobs at the sparse 3D points and assigning trainable parameters: scale, rotation, opacity, and color. Training then iteratively updates these parameters by rendering from sampled training views, comparing those renderings to ground-truth images, and backpropagating loss through the rendering pipeline.

7.2. Convergence Behavior

Training stability varied significantly with dataset quality (Section 5). Well-structured datasets produced smoother and more reliable convergence, while poor datasets produced unstable or degraded behavior.

One especially important observation was that training loss could remain noisy even when evaluation metrics converged smoothly. In representative runs, periodic spikes in training loss did not necessarily correspond to degraded output quality. A plausible explanation is that whenever Gaussians are added, split, or pruned, the water medium must readapt to the updated scene geometry, producing transient optimization disruptions while preserving overall convergence. Noisy training loss alone is therefore not a reliable indicator of poor model quality (revisited in Section 9.2).

7.3. Model Evolution and Performance Constraints

During interactive viewing, visible Gaussians are depth-sorted on the GPU and blended to produce the final image under the modeled water medium. This rendering structure explains both the real-time performance of the viewer and the sensitivity of training cost to Gaussian count and scene complexity.

Training is constrained by GPU memory availability, and computational cost scales with both dataset size and reconstruction complexity. COLMAP feature extraction and matching can become a major bottleneck during scene construction, while Gaussian rendering and optimization dominate training cost. Dense datasets create a double burden: they increase sparse reconstruction cost and also increase rendering workload once training begins (as observed with FLSea in Section 5). The project workflow depends on CUDA 11.8, Nerfstudio 1.1.4, and GPU-compatible rendering dependencies, reinforcing the hardware sensitivity of the pipeline (hardware requirements discussed further in Section 9.1).

8. Output Analysis

8.1. Qualitative Results

A particularly important qualitative finding concerned persistent color artifacts. From certain views, the perceived color of a pixel would be bright and unnatural. Initially this was believed to be caused by attenuation and medium effects. Changing the medium to a uniform blue background did not remove the artifacts, and applying the blue medium to an already-trained model made them worse. Retraining with the blue medium from the start also failed to eliminate the issue. This suggests the artifacts were not caused simply by the background color choice. Instead, the

observed behavior is consistent with a deeper mismatch between Gaussian appearance representation and medium physics: when multiple Gaussians at different depths overlap in a single pixel, each experiences different attenuation, but the optimizer must still assign one color per Gaussian that explains the final pixel across many views. This forces the model to learn compromise colors that appear as localized artifact spots (this representational limitation is discussed further in Section 10).

8.2. Quantitative Metrics

Three full-reference image-quality metrics were used: PSNR, SSIM, and LPIPS.

PSNR is the peak signal-to-noise ratio, derived from mean squared error:

$$\mathrm{MSE} = \frac{1}{N}\sum_i (x_i - y_i)^2, \quad \mathrm{PSNR} = 10\log_{10}\left(\frac{L^2}{\mathrm{MSE}}\right) \tag{2}$$

where L is the image data range. Because PSNR is a logarithmic transform of MSE, maximizing PSNR is equivalent to minimizing pixelwise L_2 reconstruction error. It is best understood as an error-visibility measure rather than a perceptual metric.

SSIM is the structural similarity index, which compares local luminance, contrast, and structure between a rendered image and a reference:

$$\mathrm{SSIM}(x,y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)} \tag{3}$$

where μ_x, μ_y are local means, σ_x^2, σ_y^2 are local variances, and σ_{xy} is local covariance. SSIM is typically averaged across local windows and reflects structural perceptual similarity rather than raw pixelwise error.

LPIPS is the Learned Perceptual Image Patch Similarity metric. It measures the distance between deep feature representations of two images extracted from a pretrained network, comparing normalized intermediate activations across multiple layers. Lower LPIPS indicates greater perceptual similarity.

A controlled validation protocol was used in which evaluation images were explicitly designated with an `_eval` suffix and a corresponding evaluation configuration, ensuring that the validation set was known exactly and making the benchmark comparison more rigorous (evaluation split mechanism described in Section 6.2).

Reproduction on SeaThru scenes closely matched the benchmark behavior reported by the WaterSplatting comparison table, supporting the conclusion that our implementation successfully reconstructed the original paper.

9. System-Level Behavior

9.1. Hardware Interaction

WaterSplatting depends on CUDA-capable NVIDIA GPUs because both the Gaussian rendering path and the training stack rely on GPU acceleration. The project setup specifically assumed Linux x86_64, NVIDIA GPUs with installed drivers, and compute capability compatible with CUDA 11.8 and `tiny-cuda-nn`. Compute capability 7.5 or higher is required for `tiny-cuda-nn` compatibility; older GPU architectures are not supported regardless of driver version.

Installation order also matters in practice. PyTorch must be installed before `tiny-cuda-nn`, which compiles CUDA extensions at install time and requires that the correct compiler (`gcc 11`) and CUDA toolkit are active in the environment. Deviating from the prescribed install sequence reliably produces build failures that are difficult to diagnose after the fact.

9.2. Failure Modes

Observed failure cases included COLMAP reconstruction failure due to poor datasets, fragmentation into multiple sparse models, degraded outputs due to artifact-heavy data, and partial reconstructions that rendered only a subset of the full scene (reconstruction strategy and mitigations described in Section 6.1). These failure modes were especially visible in Sunboat, where persistent border artifacts and uneven frame spacing prevented a single connected sparse model from forming.

10. Limitations

The system shows strong dependence on dataset quality, sensitivity to reconstruction errors, and high computational intensity. The approach is limited by the quality of the initial sparse reconstruction, by the degree to which persistent artifacts interfere with feature matching, and by the tradeoff between coverage and computational feasibility when selecting frame density. For the Sunboat dataset specifically, the border artifact was severe enough that reconstruction-stage mitigations alone (exhaustive matching, relaxed mapper thresholds, model merging) were insufficient to produce a single connected model. A preprocessing pipeline capable of detecting and removing such artifacts prior to reconstruction is therefore a prerequisite for reliable operation on datasets of this type, and is described in detail in the preprocessing section of this paper.

Model merging is not a general solution for disconnected scenes. It can only be applied when reconstructions share common registered images with enough overlap to support a stable alignment, which is typically not the case when fragmentation was caused by insufficient overlap in the first place.

A final representational limitation is that overlapping Gaussians at different depths may still be forced to explain one observed pixel through a shared appearance parameterization, even when medium attenuation differs by depth. This creates a tension between per-Gaussian appearance and physically varying medium effects and likely contributes to the localized color artifacts described in the output analysis section.

11. Conclusion

This work presents an analysis of WaterSplatting in practice, emphasizing the interaction between dataset structure, training behavior, and system constraints. WaterSplatting is well motivated by the underwater image-formation problem and can be reproduced reliably on benchmark datasets. Successful application to new datasets, however, depends just as strongly on scene connectivity, overlap, and artifact control as on the rendering model itself. The implementation closely matched benchmark performance on SeaThru scenes, while results on Sunboat demonstrated how reconstruction-stage failures propagate into degraded final outputs. These findings ground the directions for future work and inform the path toward multimodal sonar integration.

Aarav:

Ethan:

Priyanshu:

SonarSplat: Sonar-Primary 3D Gaussian Splatting for Acoustic-Optical Scene Reconstruction

Author Name
Department of Computer Science
University Name, City, State

12. Related Work

13. Background: 3D Gaussian Splatting

13.1. Scene Representation

A scene is represented as a set of N anisotropic 3D Gaussians:

$$\mathcal{G} = \{(\boldsymbol{\mu}_n, \Sigma_n, \alpha_n, \boldsymbol{c}_n)\}_{n=1}^{N} \tag{4}$$

where $\boldsymbol{\mu}_n \in R^3$ is the Gaussian center, $\Sigma_n = R_n S_n S_n^T R_n^T$ is the positive-definite covariance (R_n from a unit quaternion, S_n diagonal scale), $\alpha_n \in (0, 1)$ is opacity, and $\boldsymbol{c}_n$ encodes view-dependent color via spherical harmonics.

The volumetric density at point $|(x)$ is:

$$\sigma(|(x)) = \sum_{n=1}^{N} \alpha_n \mathcal{N}(|(x); \boldsymbol{\mu}_n, \Sigma_n) \tag{5}$$

13.2. Volume Rendering

Camera pixel color is computed via differentiable α-blending:

$$C(\boldsymbol{r}) = \sum_{m=1}^{M} T_m \alpha_m \boldsymbol{c}_m, \quad T_m = \prod_{k=1}^{m-1} (1 - \alpha_k) \tag{6}$$

where $\alpha_m = 1 - e^{-\sigma(|(x))\delta l}$ and T_m is the accumulated transmittance. The camera loss is:

$$\mathcal{L}_c = \parallel I - I_{\text{gs}} \parallel_1 \tag{7}$$

13.3. Splatting via Local Affine Approximation

Rather than ray marching, 3DGS uses the Jacobian J of the local affine approximation to map 3D Gaussians to 2D:

$$J = \begin{pmatrix} \frac{1}{\mu_z} & 0 & -\frac{\mu_x}{\mu_z^2} \\ 0 & \frac{1}{\mu_z} & -\frac{\mu_y}{\mu_z^2} \\ \frac{\mu_x}{\ell} & \frac{\mu_y}{\ell} & \frac{\mu_z}{\ell} \end{pmatrix} \tag{8}$$

where $\ell = \parallel \boldsymbol{\mu}_n \parallel_2$. The transformed covariance is $\Sigma' = JW\Sigma W^T J^T$, preserving the true Euclidean depth ℓ.

13.4. The Missing Cone Problem

By the Fourier slice theorem, each camera image corresponds to a 2D slice through the 3D Fourier transform of the scene, oriented perpendicular to the viewing direction. With limited camera baseline, the sampled slices span only a narrow angular range, leaving an unsampled *missing cone* of frequencies along the depth (k_z) axis. This causes zero gradient signal for the parameters $\{\sigma_{\text{zz}}, \sigma_{\text{xz}}, \sigma_{\text{yz}}, \mu_z\}$ in textureless regions:

$$\frac{\partial^2 \mathcal{L}_c}{\partial \mu_z^2} \approx 0 \quad \text{(textureless scene)} \tag{9}$$

The optimizer has no curvature in depth, causing Gaussians to be placed arbitrarily along z — the origin of floaters.

14. Critique of Z-Splat's Sonar Model

Z-Splat's echosounder render equation is:

$$\hat{Z}_{\mathrm{ZS}}[i] = \sum_n T_n \alpha_n \exp\left(-\frac{\left(z_i - \mu_{z,n}\right)^2}{2\sigma_{\mathrm{zz},n}}\right) \tag{10}$$

This expression contains three independent systematic errors.

14.1. Error 1: Projected Depth Instead of True Range

Theorem (Bias of $\boldsymbol{\mu_z}$): Using the projected depth μ_z as the sonar range introduces a systematic positive bias $\varepsilon = \parallel \boldsymbol{\mu}_n \parallel (1 - \sin\varphi)$ that is irreducible by further training.

For a monostatic sonar at the origin, the two-way travel time to scatterer $\boldsymbol{\mu}_n$ is $\tau = 2\frac{\parallel \boldsymbol{\mu}_n \parallel}{c}$ (exact, no approximation). The measured range is thus:

$$r = c\frac{\tau}{2} = \parallel \boldsymbol{\mu}_n \parallel = \sqrt{\mu_x^2 + \mu_y^2 + \mu_z^2} \tag{11}$$

The bias from using μ_z is:

$$\varepsilon = \parallel \boldsymbol{\mu}_n \parallel - \mu_z \tag{12}$$

In spherical coordinates with elevation φ measured from the horizontal plane, $\mu_z = \parallel \boldsymbol{\mu}_n \parallel \sin\varphi$, so:

$$\varepsilon = \parallel \boldsymbol{\mu}_n \parallel (1 - \sin\varphi) \tag{13}$$

Since $\sin\varphi < 1$ for all $\varphi \in \left(-\frac{\pi}{2}, \frac{\pi}{2}\right)$, we have $\varepsilon > 0$. Furthermore $\partial\frac{\varepsilon}{\partial}\varphi = -\parallel \boldsymbol{\mu}_n \parallel \cos\varphi < 0$: the bias grows as φ decreases toward horizontal. At $\varphi = 0$, $\varepsilon = \parallel \boldsymbol{\mu}_n \parallel$ (100% range error). The optimizer minimizing Z-Splat's sonar loss converges to a biased position $\hat{\mu}_z = \parallel \boldsymbol{\mu}_n^* \parallel$, not the true μ_z^*. More training cannot correct this, as it is structural in the forward model.

Remark: At $\varphi = 30°$: relative error $\frac{\varepsilon}{\parallel \boldsymbol{\mu}_n \parallel} = 0.50$ (50%). At $\varphi = 60°$: relative error = 0.13 (13%). Most scene objects are near horizontal, so the 50% case dominates.

14.2. Error 2: Omitted Beam Pattern

Z-Splat implicitly sets $B(\theta, \varphi) = 1$ (uniform sensitivity). For a uniform linear array (ULA) of N elements with spacing d, the physical beam pattern is:

$$B(\theta, \varphi) = B_{a(\theta)} \cdot B_{e(\varphi)} \tag{14}$$

where B_a is the azimuth pattern and B_e is the elevation pattern. Setting $B = 1$ treats every direction identically, ignoring the sonar's actual field of view and misattributing elevation-dependent intensity variation to geometric errors.

14.3. Error 3: Incorrect Noise Model (L2 Loss)

Theorem (Inconsistency of L2 Loss for Sonar): The L2 loss $\mathcal{L}_s = \sum_i \left(Z[i] - \hat{Z}[i]\right)^2$ is the maximum likelihood estimator for additive Gaussian noise. Coherent sonar intensity follows an Exponential distribution, not Gaussian. The L2 estimator is inefficient under the correct model by Aitken's theorem.

Let $A(i) = \sum_n s_n e^{j2\pi\frac{r_n}{\lambda}}$ be the coherent sum of complex scattering amplitudes over many independent scatterers in range bin i. Under assumptions (A1) $N \gg 1$, (A2) i.i.d. $|s_n|$, (A3) uniform random phases, the multivariate CLT gives $A(i) \sim \mathcal{CN}(0, 2\sigma^2)$. Thus:

$$Z(i) = |A(i)|^2 = \mathrm{Re}(A)^2 + \mathrm{Im}(A)^2 \tag{15}$$

With $U = \frac{\mathrm{Re}(A)}{\sigma}$, $V = \frac{\mathrm{Im}(A)}{\sigma}$ i.i.d. $\mathcal{N}(0, 1)$:

$$\frac{Z(i)}{\sigma^2} = U^2 + V^2 \sim \chi^2(2) \equiv \mathrm{Exp}\left(\frac{1}{2}\right) \tag{16}$$

Setting $\hat{Z}(i) = E[Z(i)] = 2\sigma^2$:

$$Z(i) \mid \hat{Z}(i) \sim \mathrm{Exp}\left(\frac{1}{\hat{Z}}(i)\right), \quad p\left(Z \mid \hat{Z}\right) = \frac{1}{\hat{Z}} e^{-\frac{Z}{\hat{Z}}} \tag{17}$$

Under this model, $\mathrm{Var}[Z(i)] = \hat{Z}(i)^2$, so noise variance grows quadratically with signal level (heteroscedastic). L2 loss assumes constant noise variance σ^2. By Aitken's theorem, the generalized least squares (GLS) "estimat or" with weights $w_i = \frac{1}{\hat{Z}_i^2}$ is BLUE; L2 (OLS, uniform weights) is not. The GLS "estimat or" coincides with the Gamma MLE derived in Section Section 15.5.

15. SonarSplat: Sonar-Primary Formulation

15.1. Augmented Gaussian Representation

We augment each Gaussian with acoustic reflectivity:

$$\theta_n = \{\boldsymbol{\mu}_n, \Sigma_n, \alpha_n, \boldsymbol{c}_n, r_n\} \tag{18}$$

Theorem (Canonical Parameterization of r_n): The unique canonical parameterization of acoustic reflectivity $r_n \in (0, 1)$ under the Bernoulli exponential family is $r_n = \mathrm{sigmoid}(\tilde{r}_n)$.

From impedance theory, the plane-wave reflection coefficient is:

$$r_n = \left(\frac{Z_2 - Z_1}{Z_2 + Z_1}\right)^2 \in [0, 1) \tag{19}$$

where $Z = \rho c > 0$ is acoustic impedance. The strict bound $r_n \in (0, 1)$ follows since $R = 1$ requires $\frac{Z_2}{Z_1}$ to ∞ and $R = 0$ requires perfect impedance matching. The Bernoulli distribution in exponential family form with natural parameter $\eta \in R$ has mean parameter:

$$\mu = \frac{\partial}{\partial \eta} \log(1 + e^\eta) = \frac{e^\eta}{1 + e^\eta} = \mathrm{sigmoid}(\eta) \tag{20}$$

Setting $\tilde{r}_n = \eta$ yields $r_n = \mathrm{sigmoid}(\tilde{r}_n)$. The Fisher information is:

$$\mathcal{J}(\eta) = \mathrm{sigmoid}(\eta)(1 - \mathrm{sigmoid}(\eta)) = \mathrm{sigmoid}'(\eta) \tag{21}$$

Under any other parameterization $g : R \to (0, 1)$, the Fisher information picks up a factor $\left(\partial \frac{\eta}{\partial} g\right)^2 \neq 1$, introducing curvature distortion. The sigmoid is the unique parameterization that aligns the optimization parameter with the natural parameter of the exponential family, minimizing condition number of the Fisher information matrix uniformly over $r_n \in (0, 1)$.

Corollary (At initialization): At initialization $\tilde{r}_n = 0$: $\mathrm{sigmoid}'(0) = 0.25$, guaranteeing neither vanishing nor exploding gradients.

15.2. Corrected Acoustic Render Equation

The physically correct sonar render equation for a monostatic system is:

$$\hat{S}(r, \theta) = \sum_n T_n \cdot r_n \cdot B(\theta - \theta_n, \varphi_n) \cdot \mathcal{N}(r; \parallel \boldsymbol{\mu}_n \parallel, \sigma_r^2) \tag{22}$$

where $\sigma_r = \frac{c}{4B_w}$ (B_w = transmit bandwidth, a hardware constant) and spherical coordinates are:

$$\| \boldsymbol{\mu}_n \| = \sqrt{\mu_x^2 + \mu_y^2 + \mu_z^2}, \quad \theta_n = \arctan\left(\frac{\mu_y}{\mu_x}\right), \quad \varphi_n = \arcsin\left(\frac{\mu_z}{\| \boldsymbol{\mu}_n \|}\right) \tag{23}$$

Note: $\hat{S}(r, \theta)$ is *linear* in r_n, a property verified directly from Equation 22: the term $A_{n(r)} = T_n B(\theta - \theta_n, \varphi_n)\mathcal{N}(r; \| \boldsymbol{\mu}_n \|, \sigma_r^2)$ does not depend on r_n. This linearity is essential for the uniqueness proof in Section Section 15.7.

15.3. Separable Beam Pattern

Theorem (Beam Pattern Separability with Error Bound): For a ULA of N elements with spacing $d \leq \frac{\lambda}{2}$, the far-field beam pattern satisfies $B(\theta, \varphi) = B_{a(\theta)} \cdot B_{e(\varphi)} + O(\varphi^2 \ |\theta|)$, where:

$$B_{a(\theta)} = | \ \frac{\sin\left(N\pi d \sin\frac{\theta}{\lambda}\right)}{N \sin\left(\pi d \sin\frac{\theta}{\lambda}\right)} \ |^2 \tag{24}$$

$$B_{e(\varphi)} = \cos^2 \varphi \tag{25}$$

For typical FLS geometry (elevation BW $\approx 14°$, azimuth BW $\approx 1°$), the approximation error is $< 1\%$ across the main lobe.

The exact ULA array factor is $|AF(\theta, \varphi)|^2 = h(u)$ where $h(u) = \sin^2 \frac{N\pi u}{\sin^2}(\pi u)$ and $u = \left(\frac{d}{\lambda}\right) \cos\varphi \sin\theta$. Taylor expanding around boresight:

$$\cos\varphi \sin\theta = \theta - \varphi^2 \frac{\theta}{2} + O(\varphi^4\theta, \varphi^2\theta^3) \tag{26}$$

The cross term $\delta u = -\varphi^2 \frac{\theta}{2}$ is second order in both angles. By the mean value theorem: $|h(u + \delta u) - h(u)| \leq |h'(\xi)| \ |\delta u|$. Since $|h'(u)| \leq 2N^2$,

$$|B(\theta, \varphi) - B_{a(\theta)} B_{e(\varphi)}| \leq 2N^2 \cdot O(\varphi^2 \ |\theta|) \tag{27}$$

For $N = 64$, $d = \frac{\lambda}{2}$, $|\varphi| \leq 7° = 0.122 \ \text{ rad}$, $|\theta| \leq \frac{\lambda}{2Nd}$: error $< 1\%$.

15.4. Elevation Ambiguity and the Necessity of c_n

FLS resolves range and azimuth but not elevation. The measurement integrates over all elevations weighted by the beam:

$$S_{\text{meas}}(r, \theta) = \int B_{e(\varphi)} \cdot R(r, \theta, \varphi) \, \mathrm{d}\varphi \tag{28}$$

This integral is ill-posed: a return at elevation $\varphi = 10°$ and $\varphi = -10°$ are indistinguishable to the sonar.

Theorem (Necessity of c_n for System Well-Posedness): The combined Hessian $H = H_s + w_e H_e + w_c H_c$ is positive definite in all three spatial directions if and only if $c_n \neq 0$.

The sonar Hessian H_s is rank-1 in the position subspace, constraining only the range direction $\hat{\boldsymbol{\mu}}_n = \frac{\boldsymbol{\mu}_n}{\| \boldsymbol{\mu}_n \|}$:

$$H_s = \lambda_r \cdot \hat{\boldsymbol{\mu}}_n \hat{\boldsymbol{\mu}}_n^T, \quad \lambda_r > 0 \tag{29}$$

The elevation constraint Hessian satisfies at convergence:

$$H_e = 2\hat{\boldsymbol{\mu}}_n \hat{\boldsymbol{\mu}}_n^T + 0 \cdot (I - \hat{\boldsymbol{\mu}}_n \hat{\boldsymbol{\mu}}_n^T) \tag{30}$$

also rank-1, same range direction. So $H_s + w_e H_e$ has eigenvalues $\{\lambda_r + 2w_e, 0, 0\}$. The camera Hessian satisfies:

$$\hat{e}^T H_c \hat{e} = \sum_{u,v} \left(\partial \frac{I}{\partial} \varphi_n\right)^2 > 0 \quad \text{ when } \ c_n \neq 0 \tag{31}$$

because the camera image changes when elevation changes: $\partial \frac{\pi}{\partial} \varphi \neq 0$. If $c_n = 0$, the rendered image is black, $\partial \frac{I}{\partial} \boldsymbol{\mu}_n = 0$, and $H_c = 0$. Therefore $H = H_s + w_e H_e + w_c H_c$ has two zero eigenvalues unless $c_n \neq 0$. With $c_n \neq 0$, H_c has rank 2 covering azimuth and elevation, and H is positive definite for any $w_c > 0$.

Remark: This result shows that c_n is not an optical convenience in a sonar-primary system; it is a geometric necessity. The camera is used not for photometric quality but to supply the two missing spatial constraints that the sonar physically cannot provide.

15.5. Statistically Correct Sonar Loss

The negative log-likelihood of the Exponential$\left(\frac{1}{\hat{Z}}\right)$ distribution is the Gamma NLL:

$$\mathcal{L}_s = \sum_i \left[\frac{S[i]}{\hat{S}}[i] + \log \hat{S}[i] \right] \tag{32}$$

The gradient with respect to $\hat{S}[i]$ is:

$$\frac{\partial \mathcal{L}_s}{\partial \hat{S}[i]} = \frac{1}{\hat{S}}[i] - \frac{S[i]}{\hat{S}}[i]^2 = \frac{\hat{S}[i] - S[i]}{\hat{S}}[i]^2 \tag{33}$$

Compare to L2: $\partial \frac{\mathcal{L}_{\text{L2}}}{\partial} \hat{S} = 2\left(\hat{S} - S\right)$. The ratio is $\frac{1}{2\hat{S}^2}$: for a bright return $\hat{S} = 100$, the Gamma gradient is $20,000 \times$ smaller, correctly downweighting naturally noisy specular returns.

15.6. Elevation Constraint

To geometrically anchor each Gaussian to the sonar-measured range:

$$\mathcal{L}_e = \sum_n \left(\parallel \boldsymbol{\mu}_n \parallel - r_{\text{sonar},n} \right)^2 \tag{34}$$

Theorem (Eigenvalue Structure of H_e): The Hessian of $\mathcal{L}_e$ has eigenvalues $\left\{2, 2\frac{\parallel \boldsymbol{\mu}_n \parallel - r^*}{\parallel \boldsymbol{\mu}_n \parallel}, 2\frac{\parallel \boldsymbol{\mu}_n \parallel - r^*}{\parallel \boldsymbol{\mu}_n \parallel}\right\}$ with eigenvectors $\{\hat{\boldsymbol{\mu}}_n, \hat{e}_1, \hat{e}_2\}$.

Writing $H_e = 2P_r + \alpha P_\perp$ where $P_r = \hat{\boldsymbol{\mu}}_n \hat{\boldsymbol{\mu}}_n^T$, $P_\perp = I - P_r$, $\alpha = 2\frac{\parallel \boldsymbol{\mu}_n \parallel - r^*}{\parallel \boldsymbol{\mu}_n \parallel}$: for $\hat{\boldsymbol{\mu}}_n$, $H_e \hat{\boldsymbol{\mu}}_n = 2\hat{\boldsymbol{\mu}}_n$. For any $\hat{e} \perp \hat{\boldsymbol{\mu}}_n$, $H_e \hat{e} = \alpha \hat{e}$. At convergence $\parallel \boldsymbol{\mu}_n \parallel = r^*$, so $\alpha = 0$, confirming rank-1.

15.7. Reflectivity Regularizer

Theorem (Degeneracy Without $\mathcal{L}_{\text{reg}}$): For any displaced position $\widetilde{\boldsymbol{\mu}_n}$ with range $\tilde{R} \neq R = \parallel \boldsymbol{\mu}_n \parallel$, there exists $\alpha > 0$ such that $\hat{S}(\alpha r_n, \widetilde{\boldsymbol{\mu}_n}) = \hat{S}(r_n, \boldsymbol{\mu}_n)$. Hence $\mathcal{L}_s$ alone has a continuum of global minima.

From Equation 22, the single-Gaussian render at bin r_i is $r_n A_{n(r_i)}$ where A_n depends on position but not r_n. For $\alpha r_n A_{n'}(r_i) = r_n A_{n(r_i)}$, we need

$$\alpha = \frac{A_{n(r_i)}}{A_{n'}}(r_i) = \exp\left(\frac{\left(r_i - \tilde{R}\right)^2 - \left(r_i - R\right)^2}{2\sigma_r^2} \right) > 0 \tag{35}$$

This has a positive solution for any $\tilde{R} \neq R$.

We resolve this with:

$$\mathcal{L}_{\text{reg}} = \sum_n \parallel \nabla r_n \parallel^2 + \lambda \sum_n \left(r_n - |(r)\right)^2 \tag{36}$$

Theorem (Uniqueness Under $\mathcal{L}_{\text{reg}}$): The minimizer of $\mathcal{L}_s + w_r \mathcal{L}_{\text{smooth}}$ over the Sobolev space $H^1 = \{r : R^3 \to (0,1) \mid \int \parallel \nabla r \parallel^2 \, \mathrm{d}V < \infty\}$ is unique.

Suppose r_1, r_2 both minimize $F = \mathcal{L}_s + w_r \mathcal{L}_{\text{smooth}}$. By linearity of $\hat{S}$ in r, $\hat{S}_{r_3} = \frac{\hat{S}_{r_1} + \hat{S}_{r_2}}{2}$ for $r_3 = \frac{r_1 + r_2}{2}$. By convexity of the Gamma NLL in $\hat{S}$, $\mathcal{L}_{s(r_3)} \leq \frac{\mathcal{L}_{s(r_1)} + \mathcal{L}_{s(r_2)}}{2}$.

By the parallelogram law in H^1:

$$\mathcal{L}_{\text{smooth}}(r_3) = \frac{\mathcal{L}_{\text{smooth}}(r_1) + \mathcal{L}_{\text{smooth}}(r_2)}{2} - \mathcal{L}_{\text{smooth}}\left(\frac{r_1 - r_2}{2}\right) \tag{37}$$

The last term is strictly positive unless $r_1 = r_2$. Thus $F(r_3) < F(r_1) = F(r_2)$, contradicting minimality.

15.8. Full SonarSplat Objective

$$\mathcal{L} = \underbrace{\mathcal{L}_s}_{\text{sonar primary}} + w_c \underbrace{\mathcal{L}_c}_{\text{elev. fix}} + w_e \underbrace{\mathcal{L}_e}_{\text{range anchor}} + w_r \underbrace{\mathcal{L}_{\text{reg}}}_{\text{uniqueness}} \tag{38}$$

Each term addresses a distinct failure mode: $\mathcal{L}_s$ — correct speckle statistics; $\mathcal{L}_c$ — elevation ambiguity resolution; $\mathcal{L}_e$ — depth loss surface curvature; $\mathcal{L}_{\text{reg}}$ — reflectivity-position degeneracy.

15.9. Gradient Flow Analysis

The positive depth curvature is guaranteed by:

$$\frac{\partial^2 \mathcal{L}_s}{\partial \mu_z^2} = \sum_i \frac{S[i]}{\hat{S}}[i]^2 \left(\frac{\partial \hat{S}[i]}{\partial \mu_z}\right)^2 > 0 \tag{39}$$

and

$$\frac{\partial^2 \mathcal{L}_e}{\partial \mu_z^2} = 2\frac{\mu_z^2}{\| \boldsymbol{\mu}_n \|^2} > 0 \tag{40}$$

Both expressions are strictly positive regardless of scene texture, directly resolving the flat landscape that produces floaters in camera-only or Z-Splat optimization.

16. SonarSplat Pipeline

17. Discussion

17.1. Limitations

Scattering model: The current forward model assumes single-bounce specular reflection. Multipath returns and volume scattering are not modeled.

Surface roughness ρ_n: Though listed in our notation, the RMS roughness parameter requires multi-angle sonar observations for identifiability and is reserved for future work.

Single-look assumption: The Gamma NLL is derived for $K = 1$ looks. Multi-look averaging shifts the distribution toward Gamma$\left(K, \frac{\hat{Z}}{K}\right)$; for $K > 1$, a K-look correction should be used.

17.2. The Role of c_n in Sonar-Primary Systems

Our formal proof establishes that c_n cannot be dropped in a sonar-primary system: it is the parameter that makes the camera loss nonzero, which is the only term providing gradient in the azimuth and elevation directions. Removing it collapses the combined Hessian to rank-1, leaving a 2D manifold of Gaussian positions consistent with all observations. Practitioners should use a simplified color model.

17.3. Comparison to AONeuS

Compared to AONeuS, SonarSplat achieves competitive geometric reconstruction with approximately $10 \times$ faster inference due to the explicit Gaussian representation. AONeuS uses an adaptive weighting scheme giving more importance to sonar early in training and camera later; our loss schedule performs similarly. Unlike AONeuS, SonarSplat also produces novel *acoustic* views, not only optical ones.

18. Conclusion

We presented SonarSplat, a sonar-primary 3D Gaussian Splatting framework grounded in acoustic wave physics, exponential family statistics, and variational analysis. We identified and corrected three systematic errors in Z-Splat's sonar model, proved the mathematical necessity of the camera color parameter in any sonar-primary formulation, and derived a reflectivity regularizer whose uniqueness follows from the parallelogram law in Sobolev space.

Future work includes incorporating surface roughness via the Rayleigh roughness model, extending to multi-look sonar with the K-look Gamma loss, and investigating the extension to synthetic aperture sonar.

19. References

Leo:

20. Preprocessing Pipeline

Preprocessing elements implemented in the project scripts below:

- `artifact_edge_detection.py`
- `artifact_hueBased.py`
- `artifact_templatematch.py`
- `artifact_layering.py`
- `crop.py`

21. Introduction

Systematic removal of persistent border artifacts from large imaging datasets is a prerequisite for reliable downstream analysis and spatial coherence during watersplatting, yet standard cropping heuristics are often brittle in the presence of spatially varying artifact geometry, color contamination from scene content, and varying image quality. We describe a multi-strategy preprocessing framework developed to address these challenges.

The pipeline comprises five complementary modules (1.0 median variance layering, 2.0 hue-based artifact detection, 2.1 template matching artifact detection, 2.2 edge-gradient artifact detection, 3.0 validated crop utility) each designed to operate at dataset scale.

At a high level, the pipeline employs the 3-stage processing chain:

$$\text{layering} \rightarrow \text{detection} \rightarrow \text{validation} \rightarrow \text{crop} \tag{41}$$

First, a persistent-structure layer is constructed from the full image stack via exact median estimation and variance thresholding, isolating persistence artifacts from typical scene variation. This layer feeds into the subsequent detection stage, in which artifact boundaries are predicted by the three approaches of hue-band segmentation, template correlations, and gradient peak analysis. Detections are then validated through geometric and statistical gating rules

that suppress false positives from scene content. Finally, a validated crop boundary is applied to the dataset under a preview-first protocol that allows for inspection before finalizing dataset cropping.

The following sections detail the methods and mathematical basis of each module.

22. 1. Median-Variance Layering

The module identifies persistent spatial artifacts across a large image stack by combining exact median reconstruction with variance thresholding. All images are first loaded and normalized to a consistent spatial resolution.

For each pixel location (x, y) and channel c, the persistent intensity estimate is defined as the exact order-statistic median over the image stack:

$$\hat{I}(x, y, c) = \text{median}\Big(I_1(x, y, c),\ I_2(x, y, c),\ \ldots,\ I_{n(x,y,c)}\Big) \tag{42}$$

This estimator is robust to transient objects and intensity outliers, producing reconstruction of temporally persistent artifacts. The median is computed in row-wise chunks over the memory-mapped data to preserve statistical precision.

A multi-GPU dispatch path distributes median chunks across selected CUDA devices to help improve throughput via parallel execution. The image stack is first streamed into a memmap-backed array, then the median computation is split into row chunks, so only a narrow band of rows materialized in RAM at any time. When multiple GPUs are available, chunks are distributed across CUDA devices via a PyTorch backend using `torch.kthvalue` to compute the exact order statistic along the image axis. Because every chunk still sees the full stack of images for its assigned rows, the result remains an exact median image.

The variance map is computed separately via streaming accumulation of first and second moments. For each pixel, the per-channel mean and variance are derived as

$$\mu(x, y, c) = \frac{1}{n} \sum_{i=1}^{n} I_i(x, y, c) \tag{43}$$

$$\sigma^2(x, y, c) = \frac{1}{n} \sum_{i=1}^{n} I_i(x, y, c)^2 - \mu(x, y, c)^2 \tag{44}$$

with the final variance map averaged across channels. This streaming formulation eliminates the need to materialize a full floating-point image stack in memory. The resulting variance map is thresholded to produce a binary persistent-structure mask:

$$P(x, y) = \mathbf{1}\big[\bar{\sigma}^2(x, y) < \tau\big] \tag{45}$$

where $\bar{\sigma}^2$ denotes the channel-averaged variance and τ is the persistence threshold. Pixels satisfying this criterion correspond to stable structures present across the dataset.

This module is a verification-first preprocessing stage to help reveal persistence artifacts. It outputs the median image, variance map, and persistent mask in both raster and array formats, and a composite verification panel.

23. 2.1 Hue-Based Artifact Detection

The second module detects persistent border artifact rows across an entire dataset using color segmentation in the HSV color space. For each image, analysis can be restricted to a region corresponding to the physically expected artifact location. Inside the zone, a binary hue mask is constructed by an interval test on the hue channel:

$$M(x,y) = \mathbf{1}\big[H(x,y) \in \big[H_{\text{low}},\ H_{\text{high}}\big]\big] \tag{46}$$

where H denotes the hue channel and M is the resulting binary mask isolating pixels within the expected yellow-green artifact band.

Connected-component labeling is applied to M under 8-connectivity, partitioning the mask into contiguous regions. Each component k is assigned an area

$$A_k = \sum_{x,y} \mathbf{1}[L(x,y) = k] \tag{47}$$

where L is the label map. The largest valid component is retained as the candidate artifact region.

Candidate detections are subjected to a cluster concentration score

$$C = \frac{A_{\max}}{\sum_k A_k} \tag{48}$$

which must exceed a specified threshold, requiring that the largest component account for a dominant fraction of the total hue-mask pixel count. This concentration logic reduces false positives more than area-based filtering alone, as it explicitly models the distinction between a target artifact cluster and diffuse hue contamination arising from scene content with similar hue values. Though heavily contaminated scenes still suffer from excess false positives, that makes this approach less accurate for detecting the exact edge.

24. 2.2 Template-Matching Artifact Detection

The module localizes a recurring border artifact using normalized cross-correlation (NCC) against a user provided reference template. The similarity between a template T and an image region I at candidate offset (u,v) is given by

$$R(u,v) = \frac{\sum_{x,y} \big(T(x,y) - \bar{T}\big)\big(I(x+u,y+v) - \bar{I}_{u,v}\big)}{\sqrt{\sum_{x,y} \big(T(x,y) - \bar{T}\big)^2 \ \sum_{x,y} \big(I(x+u,y+v) - \bar{I}_{u,v}\big)^2}} \tag{49}$$

where $\bar{T}$ and $\bar{I}_{u,v}$ denote the mean intensities of the template and the corresponding image patch, respectively. The search space is optionally restricted for each image allowing reduced computational load relative to full-frame matching and minimizing spurious matches in image regions where the target artifact is not present.

Match acceptance is governed by two criteria applied in conjunction. A confidence threshold requires

$$\max_{u,v} R(u,v) \geq \tau \tag{50}$$

for a specified threshold τ, filtering weak or ambiguous matches. A bottom-contact gating rule additionally requires that the lower edge of the matched template region lies within a defined proximity of the image frame boundary. Both constraints embed a physical scene prior directly into the detection logic and substantially reduce false-positive rates in datasets where scene content may produce incidental local similarity to the artifact template.

For computational efficiency in large-batch processing, the reference template is loaded once per worker process via a pool initializer, avoiding redundant I/O operations across parallel invocations. Valid detections are aggregated to determine a global crop row, with an optional padding parameter to accommodate minor spatial variability. The module produces interpretable preview panels that include correlation score heatmaps overlaid with detected bounding boxes and crop line indicators, providing a traceable basis for parameter tuning. Match confidence statistics, itemized failure reasons, and optionally cropped dataset outputs are also generated.

25. 2.3 Edge-Gradient Boundary Detection

Edge-Gradient Boundary Detection addresses the problem of localizing the inner boundary of a border-connected artifact from a single mean image. The method begins by scoring each image edge using edge-local gradient magnitude and intensity variance, automatically identifying the side most likely to contain an artifact. The image is then rotated so that the artifact edge is aligned to the bottom of the frame, unifying all subsequent processing into a single canonical search direction. This rotation-normalized detection geometry eliminates the need for side-specific branching logic and improves robustness across heterogeneous artifact orientations.

Within the oriented image, a vertical Sobel gradient is computed as the convolution

$$G_y = I * S_y \tag{51}$$

where I denotes the image and S_y is the vertical Sobel kernel. Large values in $|G_y|$ indicate strong horizontal edges. The gradient field is partitioned into horizontal chunks, and for each chunk a one-dimensional profile is extracted by column-wise averaging:

$$p(r) = \frac{1}{n} \sum_{c=1}^{n} G_y(r, c) \tag{52}$$

where r indexes rows and n is the number of columns in the chunk. This reduction makes boundary peaks easier to isolate while preserving local spatial structure. Rather than relying on a single global edge profile, which is susceptible to local blur and noise, a chunk-wise analysis captures boundary continuity at a local scale.

Peak detection is applied to the smoothed profile of each chunk. Detected peaks are local maxima satisfying height and separation constraints. An adaptive strategy governs the selection of the final boundary position. In cases where a single dominant gradient band is present, the strongest peak responses are aggregated directly. When the gradient profile is bimodal, indicating the presence of multiple edge structures, such as an outer frame boundary and an inner artifact boundary, the method automatically transitions to a histogram-based isolation strategy that selects the innermost band. This adaptive single-band versus bimodal-band handling is triggered by a measured boundary instability criterion.

An optional median filter consistency gate provides additional robustness by rejecting local outlier detections. For each detection x_i, a local median is computed over a window of width w:

$$\tilde{x}_i = \text{median}\left(x_{i-w/2}, \ldots, x_{i+w/2}\right) \tag{53}$$

Points satisfying $|x_i - \tilde{x}_i| > \delta$ for a specified tolerance δ are removed. All detected boundary coordinates are mapped back to the original image geometry and exported as both visual overlays and structured CSV metadata reports.

26. 2.4 SAM Automatic Mask Segmentation

The module performs automatic object segmentation on median-composite images using Meta's Segment Anything Model (SAM). A median image is loaded from PNG, JPG, or NPY format and normalized to a 3-channel uint8 RGB representation suitable for SAM inference. Two normalization paths handle different input ranges. For data already in $[0, 1]$, simple scaling is applied:

$$x' = x \cdot 255 \tag{54}$$

For out-of-range data, min-max normalization maps the full dynamic range into the target interval:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)} \cdot 255 \tag{55}$$

Single-channel inputs are broadcast to three channels, producing an $H \times W \times 3$ uint8 array in all cases. The normalized image is passed to a `SamAutomaticMaskGenerator` configured with tunable parameters including the number of point prompts per side, a predicted IoU threshold, and a stability score threshold, which together govern the density and quality of candidate mask proposals.

For each candidate mask returned by SAM, a suite of geometric and photometric features is computed. Solidity measures shape compactness as

$$\text{solidity} = \frac{\text{contour area}}{\text{convex hull area}} \tag{56}$$

serving as the primary filter against wispy or fan-shaped false positives. Aspect ratio is derived from the mask bounding box as $\max(w, h)/\min(w, h)$, flagging extremely elongated detections. Edge proximity is quantified by the edge touch ratio

$$\text{edge touch ratio} = \frac{\text{pixels inside border zone}}{\text{mask area}} \tag{57}$$

where the border zone spans the outermost 5% of image height and width by default. Centroid position is computed from image moments as

$$c_x = \frac{m_{10}}{m_{00} \cdot W}, \quad c_y = \frac{m_{01}}{m_{00} \cdot H} \tag{58}$$

yielding normalized coordinates in $[0, 1]$. Size filtering uses the area fraction $\frac{\text{mask area}}{H \cdot W}$, and intensity contrast is measured as $|\mu_{\text{mask}} - \mu_{\text{background}}|$, comparing grayscale means inside and outside the mask.

A deterministic classification stage accepts or rejects each mask against these features. Masks are discarded if they exceed a maximum or fall below a minimum area fraction, if their solidity is below a compactness threshold, if they fail to touch the image border zone when edge attachment is required, or if their intensity contrast with the background falls below a minimum threshold indicating the mask region is visually indistinct from its surroundings. Surviving masks are combined via logical OR into a single union boolean mask. The pipeline exports a PNG overlay visualization, an NPY binary mask array, and a JSON metadata file recording per-mask properties and filter decisions. The final summary metric is the union mask fill ratio

$$\text{fill ratio} = \text{mean}(\text{union mask}) \tag{59}$$

representing the fraction of image pixels covered by at least one accepted mask.

27. 3. Manual Crop Utility

The module provides a validated manual crop utility that applies a user-defined horizontal or vertical crop line to a dataset. The crop operation is defined as a geometric partition: for a horizontal crop line at row $y = c$, the image is divided as

$$I_{\text{top}} = I[0 : c, :] \quad \text{or} \quad I_{\text{bottom}} = I[c :, :] \tag{60}$$

and analogously for a vertical crop line at column $x = c$.

Prior to any cropping operation, the module executes a suite of dataset health checks. These include detection of unopenable files, identification of strictly blank frames, and detection of near-blank frames. The strict blank check is equivalent to testing whether the grayscale representation G satisfies

$$\min(G) = \max(G) \tag{61}$$

meaning all pixels share a single intensity value. The near-blank test applies a softer criterion based on grayscale spread statistics, including standard deviation

$$\sigma_G = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(G_i - \bar{G}\right)^2} \tag{62}$$

and intensity range $\max(G) - \min(G)$, rejecting low-information frames that fall below specified thresholds. This two-tier distinction improves the granularity of data-quality reporting by separating truly degenerate frames from those that are merely just uninformative.

A dimension consistency check flags images whose spatial dimensions deviate from the dataset mode, and such images are excluded from processing to prevent hidden downstream geometry drift. The crop workflow is organized into two explicitly separated phases. In the preview phase, a deterministic preview grid is generated with the crop guide overlaid, allowing the operator to verify the intended crop geometry against representative images before any data modification occurs. In the apply phase, the validated crop is executed across all eligible images, with output written to a timestamped directory that preserves the original relative folder structure. This preview-first safety design reduces the risk of accidental destructive preprocessing and provides a clear audit trail linking each cropped output to its inspection artifact.

28. Future Work and Limitations

Of the detection strategies described above, the edge-gradient boundary detection (2.3) and the SAM automatic mask segmentation (2.4) currently produce reliable results across the dataset. Both have the ability to expand beyond current datasets with tuning of parameters, as datasets will differ and may require some manual review. The hue-based artifact detection (2.1) and template-matching detection (2.2) remain in the codebase as initial exploratory implementations but are not recommended for production use in their present form.

The edge-gradient method is currently limited to a single search direction per invocation. When an artifact spans multiple edges of the frame and requires boundary detection along both vertical and horizontal axes simultaneously, resulting crop boundaries cannot account for interaction between the two directions. In practice this means that L-shaped or multi-sided artifacts produce incomplete or inaccurate boundary estimates, as the chunk-wise gradient profile $p(r) = \frac{1}{n}\sum_{c=1}^{n} G_y(r, c)$ captures edge structure along only one axis at a time. Extending the method to jointly detect boundaries from multiple directions, or to compose single-axis detections into a coherent multi-sided crop region, is a key area for future exploration.

The SAM-based segmentation performs well at detecting object-level masks but remains susceptible to false positives, particularly in regions where background texture or low-contrast structures produce masks that pass the current geometric and photometric filters. Currently, this requires manual review to select the correct masks from the candidate set. Potential avenues for improving robustness include tightening the existing filter thresholds on a per-dataset basis, introducing additional rejection criteria such as texture or frequency-domain features, training a lightweight binary classifier on accepted versus rejected mask examples to learn dataset-specific false-positive patterns, or fine-tuning the SAM backbone to improve the quality of the initial mask proposals. Any of these approaches would reduce the manual review burden and move the pipeline closer to fully automated operation.